

What is a Process?

- A process is a program that is currently running.
- It is also sometimes called a "job" especially in batch systems (older OS models).
- A process is more than just code. It also includes memory, registers, program counter, etc.
 - Program Counter (shows which instruction is running)
 - Registers (CPU's memory to keep temporary data)
 - Stack (stores function parameters, return addresses, etc.)
 - Data Section (holds global variables)
 - Heap (used for dynamic memory during program execution)
- In simple words, a program is just a file on disk, and a process is that program in action.

Example:

Let's say you installed the Discord app.

- When it's sitting there and not running, it's just a program (file on disk).
- When you open Discord, it becomes a process because it's running and doing work (like checking messages, notifications, etc.).

Program vs. Process

Program	Process
Passive (stored on disk)	Active (running in memory)
Just code	Code + execution state (memory, counter, etc.)

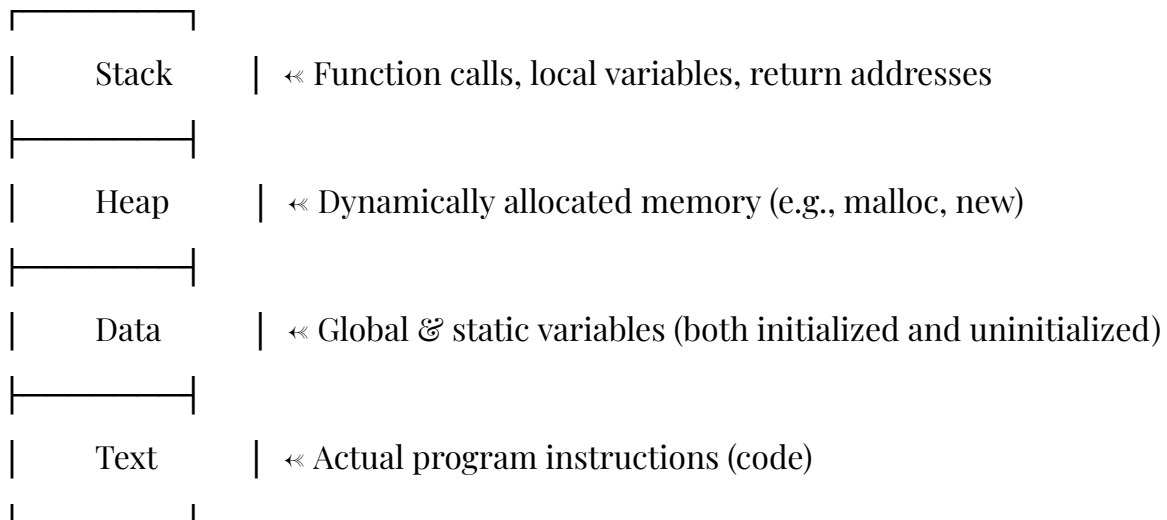
Example:

You can open the same program (like a browser) in 3 windows, these are 3 separate processes running the same code.

Memory Layout of a Process

When a program runs (i.e., becomes a process), the operating system gives it a memory space. That space is organized in a specific layout, and it typically looks like this:

↑ Higher Memory Addresses (Top)



↓ Lower Memory Addresses (Bottom)

Stack

- Stores:
 - Function parameters
 - Local variables
 - Return addresses
- Grows downward (from high memory to low)

Example:

You call a function **add(2, 3)** » arguments (2, 3) and the return address gets stored in the stack.

Heap

- Used for dynamic memory allocation
- Grows upward (from low memory to high)

Example:

In C: **int* ptr = malloc(4 * sizeof(int));**

Memory for 4 integers is allocated in the heap.

Data

Divided into two parts:

- Initialized data (e.g., **int a = 10;**)
- Uninitialized data (BSS) (e.g., **static int x;**)

Example:

Global or static variables like **int score = 100;** go into the data section.

Text

- Contains the actual executable code (machine instructions)
- Read-only for protection

Example:

The instructions of the **main()** function live in the text section.

Example

Let's say you have a simple C program:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int globalVar = 50;    // Data Section
```

```
int main() {
```

```
    int localVar = 10;    // Stack
```

```
    int* heapVar = malloc(4); // Heap
```

```
    *heapVar = 20;
```

```
    printf("Sum: %d\\n", localVar + *heapVar); // Text
```

```
    return 0;
```

```
}
```

Where does each variable go?

Variable	Section	Why?
globalVar	Data Section	It's a global variable
localVar	Stack	It's a local variable
heapVar	Heap	Dynamically allocated
main() code	Text	Part of the compiled program

Remember

Text » Data » Heap ↗ | Stack ↘

Heap and Stack grow toward each other, but should not clash with each other.

- Stack grows downward
- Heap grows upward

If a process uses too much memory, they can collide, which can crash the program or throw a memory error.